DATABASE SQL



Microsoft SQL Server

→ show databases; (keywords)

→ create database world; (db_name → DB_Name)

→ use world; (d_bname)

→ create table CSE (S_id varchar(30), S_name char(20), (attributes; allows chars, digits, s.chars)
S_marks int(2), S_branch char(10);

→ insert into CSE values ('S100', 'Ramesh', 90, 'CSE'); (a1_val, a2_val, a3_val, a4_val)

→ select * from CSE;

→ "

'S100'	'R'	90	'CSE'
'S101'	'A'	10	'CSE'
'S102'	'M'	50	'CSE'
'S103'	'A'	60	'CSE'

→ select * from CSE where S_id = 'S1004'; (selection) (only tuple printed)

→ select S_id, S_marks from CSE; (→ projection) only column printed

Transac. control lang. (TCL)

(Data def. lang DDL
Dat manupula. lang DML
Dat control lang DCL)

→ update CSE set S_marks = 100 where S_id = 'S1004'; (keyword, attribute)

→ select * from CSE; (given updated values (or) table)

→ delete from CSE where S_id = 'S1001'; (change)

→ select * from CSE; (11)

DDL	DML
→ create	select
→ Alter	insert
→ drop	update
→ commit	delete
→ Rename	

update suntax/commands

update T_name set Attribute/Filedname where Condition

for delete:

delete from T_name where condition

Diff. b/w delete & drop.

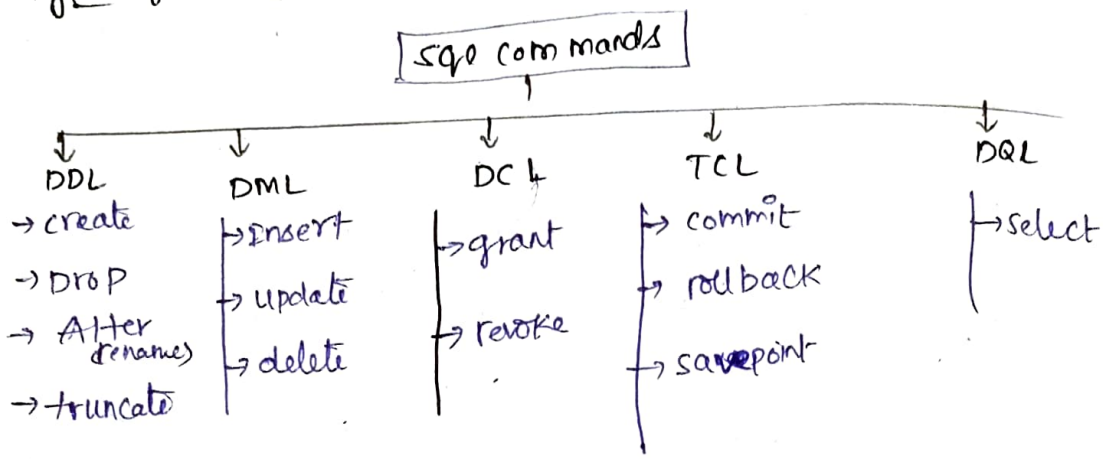
→ desc table_name; (describe) (print ← Available attributes)

→ show tables; (print all tables DB)

→ update used to change domain value

SQL commands: are ~~used~~ instructions used to communicate with DB. It can perform various tasks like create table, add, drop data from the tables etc.

Types of SQL Commands :- There are 5 types of commands



① Data def. lang (DDL) :-

DDL changes structure of ~~DB~~ table & perminantly save all change in DB

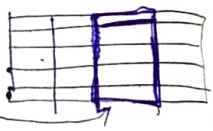
a) CREATE : used to create a new table in DB

syntax: create ~~to name~~ table t_name (column_name datatypes);

Ex:

b) DROP: used to delete both structure & record stored in table

syntax: drop table table_name;



Ex:

c) ALTER : used to alter existing column (or) to add a column in the table.

for adding :-

syntax :- alter table t_name ADD column_name (data type) column_definition;

for modification :-

syntax: alter table t_name MODIFY (column_def.s ...);

Ex: alter table Employee ADD (Address varchar(20));

alter table Employee MODIFY (name varachar(20));

d). Truncate :- used to delete all rows & free space from table

syntax: truncate table t_name;

Ex: truncate table employee;

② Data manipulation language (DML) :-

DML commands are used to modify DB. it can't be permanently save all changes in DB

a) Insert :- used to insert data into row of table

syntax: insert into t_name
values (value1, v2, v2 ... valueN);

Ex:

b) Update :- used to update / modify the value of a column in table

syntax:- update t_name SET [column-name1 = V1 , .. colun_name N = VN];

Ex:

c) delete :- used to remove one (or) more row from a table,

syntax: delete from t_name, [where condition] ;

Ex:

③ Data control lang (DCL) :- used to grant & take back the authorities / permission of user from any DB.

a) Grant :- used to give privilege to access the DB.

Ex: grant select, update on my_table to some_user, another_user;

b) Revoke :- used to take back permissions from users

Ex: revoke select, update on my_table from user1, user2 ;

④ Transaction Control language (TCL) :-

→ TCL commands can only use with DML commands like insert, delete & update only

→ These operations are automatically committed in DB, ∴ they can't be used while creating tables/ dropping them (commands)

but some other Commands are there

a) Commit :- used to save all transaction to the DB

Syntax: commit;

Ex: delete from customers where age = 25;
commit;

b) Rollback :- used to undo transactions, that haven't already been saved to DB.

Syntax: Rollback;

Ex: delete from customers where age = 25;
rollback;

c) Savepoint :- used to roll the transaction back to a certain point without rolling back the entire transaction

Syntax: Savepoint savepoint_name;

⑤ Data query lang (DQL) :- used to fetch the data from the DB.

a) select :- (only 1 command is there)

It is same as projection operation of relational algebra used to select an attribute based on condition described by where clause.

syntax :- select expressions
from table name
where conditions

Ex: select emp_name → this attribute will be printed
from employee
where age > 20

delete used to delete tuple, in a table

Drop used to delete table & column

σ → selection
π → projection
ρ - rename
Set operations { ∪ → union, ∩ → intersect, × → cross join, − → except

union, intersect } select column_name from R1 union select column_name from R2

joins → space. Ex: select * from R1 cross join R2;

S_id	S_branch
1	CSE
2	CSE
3	civil
4	mech

S_id	S_branch
1	CSE
3	civil
4	mech
5	EEE

perform set operations on these 2 tables

based on requirement, we can combine

& or not } instead we are using where

joins
- inner join (or) (simple join)
  - theta based condi.
  - Natural/equi (for printing common tuples) — equal symbol must be there
- self join
- outer join
  - left OJ
  - ROJ
  - FOJ
    - → command & syntax
    - → Ex.
    - → o/p

Aggregate
- count
- Avg
- min
- max
- sum

Rename: used to rename the database objects

Syntax to rename table:-

alter table table_name rename to new_table_name;

Syntax to rename column:-

alter table t_name rename column old_name to new_name.

Sql joins :-

- inner join
- left join
- right joint
- full join

Employee

e_id	e_name	e_salary
1	Anil	1000
2	Kanaka	2000
3	jaggu	3000
4	gana	4000

project

e_id	dept
1	CSE
2	ECE
3	mech

① **inner join** :- It returns the combination of all rows from both the tables where condition satisfies.

Syntax: select t1.colum1, t1.col2..., t2.col1, t2.col2.. from t1 inner join t2 on t1.matching_col = t2.matching_col
(t1.matching_col = t2.matching_col : condition)

o/p:

e_name	dept
Anil	CSE
Kanaka	ECE
jaggu	mech

Ex: select employee.name, project.dept from employee inner join project on employee.e_id = project.e_id

② **left join** :- returns all the values from left table & matching values from right table, if there is no matching, it will return null.

Ex: select employee.name, project.dept from employee left join project on employee.e_id = project.e_id

o/p:

e_name	dept
Anil	CSE
Kanaka	ECE
jaggu	mech
gana	null

Right join: returns all values from right table & matching values from left table, if there is no matching values it will return as zero

Ex: " right join "

o/p:

e_name	dept
Ani)	CSE
kanaka	ECE
jaggu	mech

full join: returns all the rows from both the tables & it puts null on the place of matches not found.

Ex: select employee.ename, project.dept from employee full join project on employee.eid = project.e_id.

o/p:

e_name	dept
Ani	CSE
kanaka	ECE
jaggu	mech
gana	null

SQL aggregate funcs:-

* These funcs are used to perform calculations on multiple rows of a single column, & return a single value & also used to summerize the data.

① **Count function:-** used to count no. of rows in db table. It can work on both numeric & non-numeric data types.

→ count(*) considers duplicate & null

Syntax: count(*) (or) count([ALL|DISTINCT] expression)

Ex: count(): select count(*) from product_mast; (product_mast → tablename) o/p ⇒ 10

Ex: Count with where

Ex: select count(*) from product_mast where rate>=20

o/p: 7

Ex: count() with distinct :-

```
Select count( Distinct company) from product_mast;
```

o/p: 3

Ex: count() with group by :- w.r.to

```
select count(*) from product_mast group by company;
```

o/p: com1 5
com2 3
com3 2

Ex: count() with having :-

```
// having count(*)>2
```

o/p: com1 5
com2 3

2) Sum function :- used to ~~select~~ calculate the sum of all rows of a single column., it can be work on numeric fields only

Syntax: sum() (or) sum([all | distant] expression)

Ex: sum()

```
select sum(cost) from product_mast;
```

o/p → 670

Ex: sum with where

```
Select sum(cost) from product_mast where Qty>3;
```

o/p: 320

Ex: sum() with group by :-

```
Select sum(cost) from product_mast where Qty>3 group by company
```

(key to be there)

o/p: com1 150
com2 170

Ex: sum() with having :-

```
select company, sum(cost) from product_mast group by company
having sum(cost)>=170;
```

o/p: com1 335
com3 170

3) Avg function :- used to calculate the avg. of numeric type only.

Syntax:- Avg() (or) Avg(n)

Ex:- select avg(cost) from product_mast ; o/p :- 67.00

4) max function :- used to find the max. value of all the rows of a single column.

Syntax:- max() (or) max(n)

Ex:- Select max(cost) from product_mast ; o/p :- 30

5) min function :- used to find the min. value of all the rows of a single column.

Syntax:- min() (or) min(n)

Ex:- select min(cost) from product_mast ; o/p :- 10

Self join :- or (recursive join) :-

It is used to join a table to itself, joining condition is based on column of a same table. (assume same table as 2 tables)

Syntax:-

Select a.column

emp → table name

E_Id	E_name	M_ID
10	robo	30
20	chitti	40
30	sana	10
40	vasi	20

Q1:- findout the employee name & his manager name

query:- select e1.ename, e2.e_name from emp e1, emp e2 where e1.e_id = e2.m_id

query:- select e1.ename as Emp_name, e2.e_name as man_name ;

o/p:-

Emp_name	man_name
sana	robo
vasi	chitti
robo	sana
chitti	vasi

o/p:-

e_name	e_name
sana	robo
vasi	chitti
robo	sana
chitti	vasi

Keys :- It is an attribute (or) set of attributes.

→ super key :- (having unique values) (or) unique attribute

E_id, E_adhar. E_id & E_name — ① (emp_id, emp_name)

E_adhar & E_name — ②

→ candidate key :- similar to p.k without any repitition

Eg: {E_id, E_adhar}

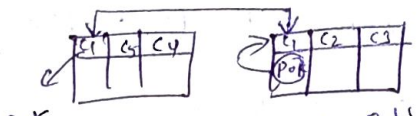
(emp_phno, emp_licno)

for employee table

→ Primary key :- (Employee_id)

Eg: E_id, E_adhar

→ foreign key :-



(Department_id)

→ Alternative key :- Except primary keys

→ Composite key :- if we don't have primary key & candidate key (Remaining key (or) keys called)

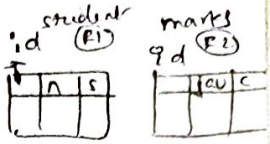
Eg: E_name & E_salary & E_contact

→ unique key :- Similar to primary key & accepts null value

E_id	E_name	E_salary	E_mail	E_adhar	E_contact
101	Anil	950	A.k.com		93916
102	Kanaka	500	xy@.com		1567
103	jagadi	600	935@.com		9910
104	Gana	700	null		11110
105	gopal	750	@mail.coma		NULL

schema view —
abstract view —
Instance view —
} views

view — virtual data from 2 (or) more tables
(or) pure
↳ relational



Relation model
↓
to access (or) to retrieve the data from relational schema
↓
theoretical knowledge — practical knowledge
↓
Relational algebra (procedural) — Relational calculus (Non-procedural)
↓
SO UO BO E/D — TRC. — DRC

expression :- → {t | p(t)}
t → o/p
p(t) → condition

DRC :- {<col1, col2, ...> / P<col1, col2, ... coln>}
<col1, col2, ...> → o/p
P<col1, col2, ... coln> → condition

Commands in sql :-

① select :- statement used to select data from DB.

Syntax: select col1, col2, ... from table_name;
(or)
select * from table_name; (for getting all cols)

② where :- where clause is used to filter the records based on a condition

Syntax: select col1, col2... from t_name where Condition;

③ sql and, (or) & not operators :-

The where clause can be combined with and, or & not operators.

* and, or operators, used to filter records based one more than one condition.

and :- operator displays a record if all conditions seperated by and are true.

suntax: select col1, col2... from t_name where condi1 and condi2 and condi3...

Ex: country = 'india'

or :- operator displays a record if any of the conditions seperated by OR is true.

syntax: select col1, col2.. from t_name where condi1 (or) condi2 or condi3...

Not: operator displays a record if condition is not true.

syntax: select col1, col2.. from t_name where not condition;

Combining and, (or), not :-

syntax:

select * from customers where ~~conty~~ country = 'germany' and (city = 'berlin' or city = 'muchen'); → parentheris imp.

meaning :- select all fields from customers where country is germany & city must be either berlin (or) munchen.

syntax: select * from customers where not country = 'germany' and not country = 'USA';

meaning :- selects all fields from customers where country is not germany & not USA

(4) order by : keyword is used to sort the result set in ascending (or) descending order (table)

bydefault, it will perform ascending order (or) use ASC

for descending order :- use DESC

syntax: ① type
select col1, col2, .. from t_name order by col1, col2..

ASC | DESC :
(or)

cols will print

col will sorted

② type
|| col1. ASC, col2 DESC :

⑤ Null : a field with no value

is null :- operator used to test for empty values

syntax: select col_names from t_name where col_name is null ;

is not null : operator used to test for non-empty values

syntax: select col_names from t_name where col_name is not null ;

like

⑥ wildCard : character used to substitute one (or) more chars in a string. It is used with - like operator, like operator is used in where clause - to search for a specified pattern in a column.

using % wildcard :-

Ex: select * from Customers where city like 'ber%' ;

meaning : selects all customers with a city starting with "ber"

→ || '%.es%' ; meaning : selects all customers with a city containing the pattern "es".

using the _ wildcard :-

Ex: select * from Customers where city like '_ondon' ;

meaning : selects all customers with a city starting with any char. followed by 'ondon'.

Ex: select * from customers where contactname like 'a_%o' ;

meaning :- select all customers with a Contact name that starts with "a" & ends with "o".

⑦ between: operator selects values within a given range. values can be numbers text (or) dates.

→ for ~~this~~ between, begin & end values are included.

Ex: select * from products where price b/w 10 and 20;

meaning:- selects all products with price b/w 10 & 20 (included)

Not between:

Ex: select * from products where price not b/w 10 and 20;

selects products outside the range of 10 and 20

b/w within:

Ex: select * from products where price b/w 10 and 20 and categoryID NOT in (1,2,3);

11, don't display products with a categoryID of 1,2,(or)3

b/w text values:-

Ex: select * from products where productname b/w (not b/w) 'Carnarvon Tigers' and 'MdG' order by product name;

meaning:- select all products with a product-name b/w (not b/w) 'Carnarvon Tigers' and MdG

b/w Dates:

Ex: select * from orders where orderdate b/w (not b/w) #07/01/1996# and #07/31/1996#;

(M/D/Y)

selects all orders within order date b/w (not b/w) 01-july-1996 & 31-july-1996.

⑧ In: operator allows us to specify multiple values in a where clause

Ex: select * from Customers where Country in ('germany', 'france', 'UK');
(or) Not in

meaning :- selects all customers that are located in 'germany', 'france' (or) 'UK'.
(or) not

Ex: select * from customers where country in (select Country from suppliers);
selects all customers that are from countries as suppliers

(9) aliases :-
As keyword is used to give aliase names for tables, cols during the execution of Query.

Ex: select CustomerID as ID, customername as Customer from Customer;
(+name)

Ex: select col_name as alias_name from table_name;
Ex: select col_names from table_name as alias_name;

(10) Exists operator :-
operator is used to test for the existence of any record in a subquery. It return true if the subquery returns one/more records

Ex: select supplier_name from suppliers where exists (select product_name from products where products.supplier ID = suppliers.supplierID and price < 20);
(returns true a lists the suppliers with a product price less than 20)

price = 22
price equal to 22

(11) Sql Any & All :-

Any & All operators allow you to perform a Comparison b/w a single Column value & a range of other values

Any operator :

→ returns a boolean value as result
→ returns true if condition satisfied.

operator must be a standard Comparison operator (=, <>, != , >, >=, <, <=).

All operator :

→ return a boolean value as a result.
→ "
→ It is used with select, where & having statements.

All syntax : select all col_name(s) from table_name where condition ;

select col_name(s) from table_name where col_name operator All (select col_name from table_name where Condition);

Any syntax :- select col_name(s) from table_name where col_name operator Any (select col_name from table_name where condition);

Ex :
select .

(12) Comments : used to explain the section of sql statement

Comments aren't supported in microsoft access DB.

single line Comments :- Any text b/w -- and the end of the line will be ignored (will not be executed)

Ex : ① -- select all → comment
select * from Customers ;

② select * From Customers -- where city = 'Berlin' ;

Multiline Comment :- starts with /* & ends with */

Ex:- /* */

select * from customers;

(13) Having :- clause was added to sql because where keyword can't be used with aggregate funcs.

Syntax:- select col_name(s) from t_name where condition group by col_name(s) having condition order by col_names;

Ex:- select count(CustomerID), country from customers group by country having count(CustomerID) > 5 order by count(CustomerID) DESC;

→ Lists no. of customers in each country, sorted high to low countries with than 5 customerIDs

(IV) Constraints :-

Constraints can be specified when table is created (or) after created with the alter table statement.

Syntax:- create table t_name(
col1 datatype constraint,
col2 " ,
col3 " ,
);

→ constraints are used to specify rules for data in a table to maintain accuracy & integrity of data

following constraints are commonly used in sql :-

a) not null Constraint :-

by default, a column can hold null values

→ not null constraint enforces a column to not accept NULL values.

→ we can't insert a new record (or) update a record without adding a value to this field.

Ex: Create table person (
ID int Not null
lastname varchar(25) Not null,
firstname varchar(25) not null,
Age int);

→ ensures the ID, lastname & f_name cols willnt accept null values. when a person table created.

using alter table statement :-

Ex: alter table persons
alter column age int Not null;

→ persons table is already created col Age value can't be Null

b) unique Constraint :-

→ ensures that all values in col are different.

→ both unique & P.K constraints provide a guarantee for uniqueness for a col (or) set of cols.

→ we can hava many unique constraints per table but only one P.K Constraint per table.

on create table:

Create table persons (
ID int not null unique, l_name varchar(255) not null,
f_name varchar(255), Age int);

, unique (ID)
we can also write here also instead of writting here

to define a unique constraint on multiple cols. :-

Create table persons(
ID int not null,
l_name varchar(255) not null,
f_name varchar(255), Age int, constraint UC_person unique (ID, l_name));

on alter table:

Eg: alter table persons add unique (ID);

→ to create a unique constraint on the 'ID' col. when the table is already created.

Eg: alter table persons add constraint UC_person unique (ID, L-name);

unique constraint

→ " multiple cols

DROP a unique constraint :-

Eg: Alter table persons drop index UC_person;

c) primary key constraint :-

→ This constraint uniquely identifies each record in a table

→ P.Ks must contain unique values, can't contain null values

→ A table can have only one P.K.

P.K on create table :-

Eg: Create table persons(

ID int not null,

L-name varchar (255) not null,

F-name varchar(255), Age int primary key (ID));

→ Creates P.K on ID col. when "persons" table created

To define a P.K constraint on multiple cols :-

Create table persons (

ID int not null, L-name varchar(255) not null,

F-name varchar(255), Age int, constraint PK_person

primary key (ID, L-name)); → there is only one P.K but P.K value is made up of 2 cols

on alter table :-

create P.K constraint on ID col. when the table is already created.

Eg: alter table persons add primary key (ID);

to define P.K Constraint on multiple cols :-

Eg: alter table persons (f_name)
add constraint PK_person primary key (ID, f_name);

→ To drop P.K Constraint :-
alter table persons drop primary key;

d) foreign key constraint :-
This constraint used to prevent actions that would destroy links b/w tables

→ A f.k is a field, that refers to p.k in another table.

→ The table with f.k is called child table
|| p.k || parent (or) reference table.

on create table :- (orders, persons tables)
creates a f.k on personID col when orders table is created

Eg: Create table orders(
orderID int not null,
ordernumber int not null,
personID int,
primary key (orderID),
foreign key (personID)
references persons (personID)
);

f.k Constraint on multiple cols :-

||
Constraint FK_personorder (foreign key)
foreign key (personID)
references persons (personID)
);

on alter table :-

To create FK on col. personID when orders table is already created

Eg: alter table orders
add foreign key (personID)
references persons (personID);

If multiple cols. :-

Ex: alter table orders
add constraint FK_personorder
foreign key (personID) references persons (personID);

Drop a foreign key constraint :-

alter table orders
drop foreign key FK_personorder;

e) Check constraint :-

Constraint used to limit the value range for a col. It will allow only certain values for this column.

on creat table :-

Ex: create table persons (
ID int not null,
l_name varchar(255) not null, F_name varchar(255)
age int,
check (Age>=18));

→ ensures that the age of a person must be 18 or older.

→ check constraint on multiple columns

Ex: create table persons (
ID int not null,
l_name varchar(255) Not null,
F_name varchar(255), Age int
city varchar (255),
Constraint CHK_person Check (Age>=18 and
city = 'sandnes'));

on alter table :- to create a check constraint on the column when the table is already created

Ex: alter table persons
add check (Age >= 18);

// multiple cols :-

Ex: alter table persons
add constraint chk_personAge
check (Age >= 18 and city = 'sandnes');

Drop a check constraint :-

Ex: Alter table persons
Drop ~~constraint~~ (or) check chk_personAge;

(f)

Default Constraint :-

This constraint used to set a default value for a col. if the default value will be added to all new records (rows) if no other value is specified.

on create table :-

→ sets a default value for the city col, when persons table is created

Ex: create table persons (
ID int not null
L_name varchar(255) Not null,
F_name varchar(255), age int
city varchar(255) default 'sandnes');

→ this constraint can also be used to insert system values, by using func.s like getdate() :-

ex: create table orders (
ID int not null,
order number int not null,
orderdate date default
getdate());

on alter table :-

to create default constraint on the city col. when table already created.

Ex: Alter table persons
alter city set default 'Sandnes';

Drop a default constraint :-

Ex: alter table persons
alter city drop default;

(9)

Create Index :-

statement is used to create indexes in tables.

→ indexes are used to retrieve data from the DB. more quickly than otherwise.

Create index syntax :-

Create an index on a table, duplicate values are allowed.

Syntax: create index index_name
on table_name (col1, col2, ...);

Create unique Index Syntax :-

" , duplicate values are not allowed

Syntax: create unique index Index_name
on table_name (col1, col2, ...);

① creates an index named "idx_lastname" on the "lastname" col in the "persons" table

Eg: Create index idx_lastname
on persons (lastname);

(2) creates an index on a combination of cols., we can list the col names with in parenthesis, separated by commas.

Eg: Create index idx_pname
ON persons (LastName, FirstName);

Drop index : used to delete an index in a table

syntax

Eg: Alter table table_name
Drop index index_name;

Date data types :-

mysql have the following data types for storing a date or time value in the DB.

* Date (format YYYY-MM-DD)
* Datetime (format: YYYY-MM-DD HH:MI:SS)
* Timestamp - "
* year → format (YYYY (or) YY)

Eg: select * from orders where Orderdate = '2008-11-11';

select records with an orderdate of '2008-11-11' from above table

Views in sql :-

→ view in sql consider as a virtual table, it contains rows & cols based on certain condition.

Creation view :-

Syntax:
```
Create view view_name as
select col1, col2...
from t_name
where condition;
```

Ex: creating view from a single table

```
Create view detailsview AS
select name, addres
from student_details
where STU_ID<4;
```

compare

Ex: creating view from multiple tables

Ex: Create view marksview as (view name)
select student_detail.name, student_detail.addres, student_marks.marks
from student_detail, student_mark ⇒ t_name
where student_detail.name = student_marks.name;

Ex: deleting view :-

Syntax: drop view view_name;

Ex: drop view marksview;

Case :

The Case expression goes through conditions & returns a value when 1st condition is true, if condition isn't true, it returns the value in the else clause.

If there is no else part & no conditions are true it returns null.

Syntax:
```
case when condition1 then result1
     condition2      result2
     :               :
     Else result
End;
```

Ex:-

```
select orderID, quantity,
Case
    when Quantity > 30 then 'The quantity is g.T 30'
    when Quantity = 30 then '        "        equal to 30'
    Else 'The quantity is under 30'
End as quantity_text
from orderdetails;
```

→ returns value when - 1st condition is true.

```
order by
( case
  =
  End);
```

} using order by remains code same. as in previous example

Auto increment :- → defined as a field i.e mainly used to generate a unique number for each record being added to a table at any instance.

→ in general terms, used for a P.K col. because a P.K is supposed to be unique & not null, thus auto increment makes it easy for the developers to automatically generate a unique no. for all records.

Ex:- create table Dataflair_emp → t_name(
ID int not null auto_increment,
name varchar(50),
location varchar(50),
primary key (ID));